1. react로 쇼핑몰 만들기

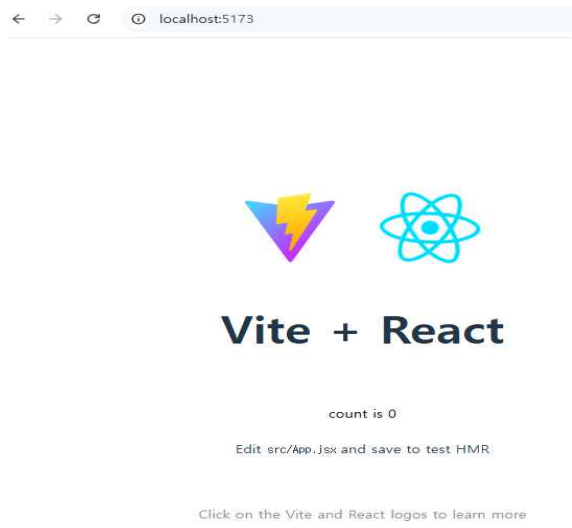
<table>
<tr><th>순서</th><th>작업환경</th><th colspan="2">명령어</th></tr>
<tr><td rowspan="2">1</td><td rowspan="2">vs code 열기</td><td>

프로젝트 생성
npm create vite@latest shop

shop폴더로 이동
cd shop

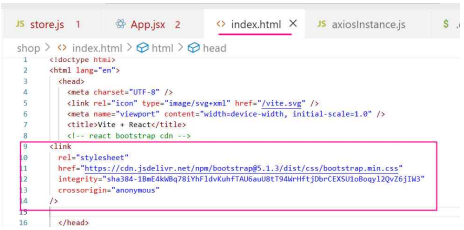
node package install하기
npm i

실행하기
npm run dev

</td><td rowspan="2"> 
</td></tr>
<tr><td>

프론트서버 종료 단축키
Ctrl + C

</td></tr>
</table>

2	App.jsx 파일 열기	3. 이미지 아래 imoprt 부분 삭제 import reactLogo from './assets/react.svg 'import viteLogo from '/vite.svg 4. App 사이에 있는거 다 삭제 -> 만 남기기 5. App.css / index.css 안에 있는 내용 모두 지우기
3	에디터에서 터미널켜고 Terminal > new Terminal // 단축키 Ctrl +` 라이브러리 설치할때는 프론트 서버 종료 Ctrl + C	React-Bootstrap 라이브러리를 설치해서 이용하기 구글에서 react bootstrap 검색 https://react-bootstrap.github.io/ 가서 get started 버튼클릭 1. 터미널에서 **npm install react-bootstrap bootstrap**
4	index.html > <head> 태그 사이에 넣기 	react bootstrap cdn link 걸기 <!-- react bootstrap cdn --> <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css" integrity="sha384-1BmE4kWBq78iYhFldvKuhfTAU6auU8tT94WrHftjDbrCEXSU1oBoqyl2QvZ6jIW3" crossorigin="anonymous" />
5	react bootstrap > 왼쪽 서치창 > navbar 검색	1. 서치란 https://react-bootstrap.netlify.app/docs/getting-started/introduction
6	App.jsx 파일에서 1. import 하기 2. return 안에 <></> 안에 html 태그 가져오기	import {Button, Navbar, Container, Nav} from 'react-bootstrap' function App() { return (<> <Navbar bg="dark" variant="dark">> <Container>

<table>
<tr><td></td><td></td><td>

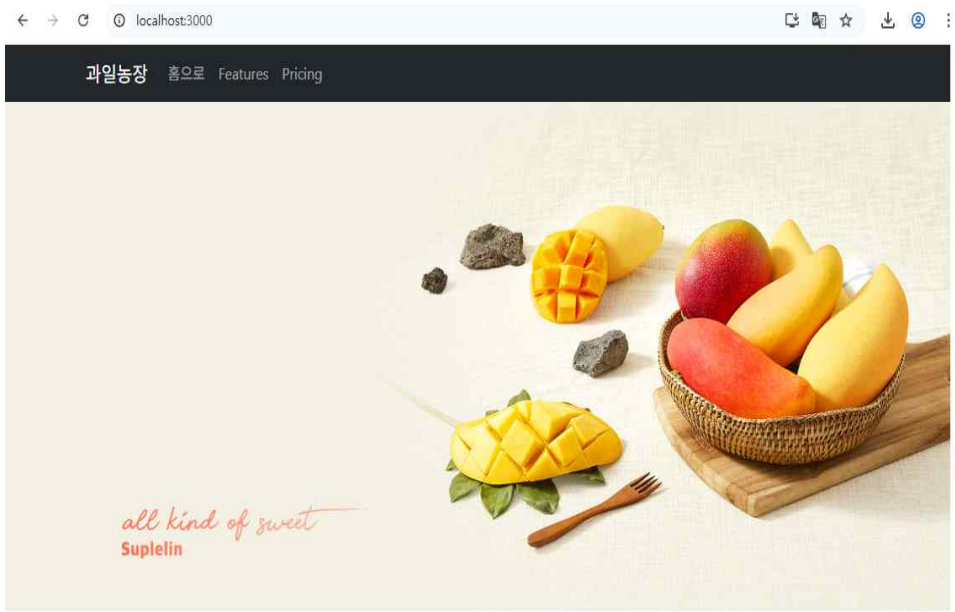
```
                <Navbar.Brand href="#home">과일농장</Navbar.Brand>
                <Nav className="me-auto">
                  <Nav.Link href="#home">홈으로</Nav.Link>
                  <Nav.Link href="#detail">상세페이지</Nav.Link>
                  <Nav.Link href="#cart">장바구니</Nav.Link>
                </Nav>
              </Container>
            </Navbar>
          </>

      </div>
    );
}
export default App;
```

</td></tr>
<tr><td rowspan="3">7</td><td rowspan="3">터미널에서</td><td>npm run dev // 로컬 호스트에서 확인
브라우저가 안뜨면 http://localhost:5173/ 열어서 확인</td></tr>
<tr><td>http://localhost:5173/</td></tr>
<tr><td>
</td></tr>
</table>

2 이미지 넣는 법 & public 폴더 사용

<table>
<tr><th>순서</th><th>작업환경</th><th colspan="2">명령어</th></tr>
<tr><td rowspan="5">1</td><td rowspan="5">slider.jpg 이미지 넣기

img폴더(다운받은)
public 폴더 아래 복사해두기

public >img</td><td colspan="2">1. img.zip 파일 제공
2. 다운로드 받은 img.zip파일 압축 풀기
3. react 작업 폴더 > public > img 폴더 째 넣기
정적파일은 public 폴더에 넣기 (경로체크 : img폴더 안에 img폴더로 하위경로가 아닌지 확인 public> img)</td></tr>
<tr><td>public/img/slider.jpg 에 이미지 갖다넣기
App.js 파일에서 navbar 밑에</td><td rowspan="4"> 

1 2 3</td></tr>
<tr><td> </td></tr>
<tr><td>index.css파일에 추가</td></tr>
<tr><td>.slider {
height:500px;
background-image: url('../public/img/slider.jpg');
background-size: cover;
background-position: center;
margin-bottom: 30px;
}</td></tr>
<tr><td rowspan="2">2</td><td rowspan="2">상품을 가로로 화면을 3등분하고 싶으면</td><td>App.js 파일 안에 (slider 아래쪽에)</td></tr>
<tr><td><div className="container" style={{textAlign:"center"}}>
 <div className="col-md-4">1</div>
 2
 3
 </td></tr>
</table>

3	3등분 안에 컨텐츠 넣기 코드 정렬 : Alt + shift +f

<img src="/img/fruit1.jpg" width="80%" />
<h4>상품명</h4>
상품정보
가격

<img src="/img/fruit2.jpg" width="80%" />
<h4>상품명</h4>
상품정보
가격

<img src="/img/fruit3.jpg" width="80%" />
<h4>상품명</h4>
상품정보
가격

 | 터미널 npm run dev
http://localhost:5173/
 |

3. 코드가 길어지면 객체 데이터 import/export 처리 → 상품 컴포넌트 만들기

순서	작업환경	명령어
1	src폴더 > **db 폴더만들고** > **fruit.js** 만들기 export default 다른곳에서 사용가능하게 하기	**fruit**.js 파일 안에 붙여넣기 let data = [{ }, { }, { }] let data = [{ id: 1, title: "수박", imgUrl: "img/fruit1.jpg", content: "당도선별 프리미엄 고당도 하우스수박 5~6kg", price: 29000, }, { id: 2, title: "참외", imgUrl: "img/fruit2.jpg", content: "산지직송 성주 달콤참외 3kg", price: 16900, }, { id: 3, title: "사과", imgUrl: "img/fruit3.jpg", content: "고씨네농장 고당도 청송사과, 햇부사 5kg", price: 13000, }, { id: 4, title: "바나나", imgUrl: "img/fruit4.jpg", content: "델몬트 필리핀 바나나 6kg", price: 15000, }, { id: 5,

```
    title: "딸기",
    imgUrl: "img/fruit5.jpg",
    content: "설향 딸기, 킹스베리 1박스",
    price: 14500,
  },
  {
    id: 6,
    title: "오렌지",
    imgUrl: "img/fruit6.jpg",
    content: "캘리포니아 네이블 오렌지 4kg",
    price: 17000,
  },
  {
    id: 7,
    title: "토마토",
    imgUrl: "img/fruit7.jpg",
    content: "행복한 농부 완숙찰토마토 5kg",
    price: 20000,
  },
  {
    id: 8,
    title: "포도",
    imgUrl: "img/fruit8.jpg",
    content: "팜앤프룻 프리미엄 아삭한 애플청포도",
    price: 25000,
  },
  {
    id: 9,
    title: "망고",
    imgUrl: "img/fruit9.jpg",
    content: "프리미엄 제주애플망고 1박스",
    price: 18500,
  },
];

export default data;
```

<table>
<tr>
<td rowspan="2">2</td>
<td rowspan="2">App.jsx 파일에서
import fruit.jsx
useState 만들기</td>
<td colspan="2">

```
import data from './db/fruit';
import { useState } from 'react';

function App(){
  const [fruit] = useState(data);
  console.log(fruit[0].price);
}
```

</td>
</tr>
<tr>
<td>

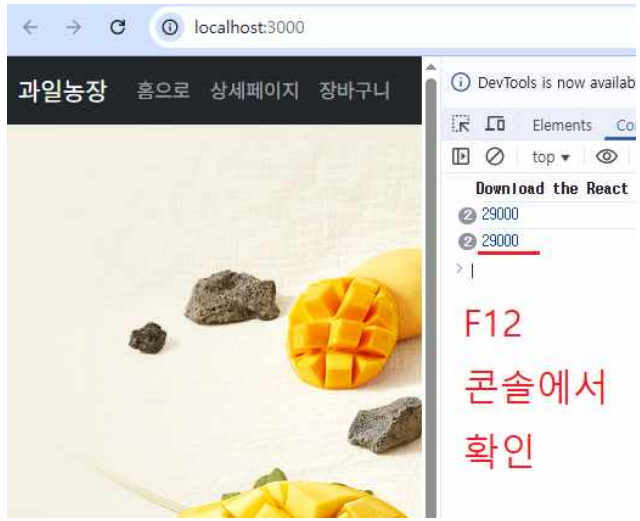
```
<div className="container" style={{textAlign:"center"}}>
        <div className="row">

          <div className="col-md-4">
            <img src="/img/fruit1.jpg" width="80%" />
            <h4>{fruit[0].title}</h4>
            <p>{fruit[0].content}</p>
            <span>{fruit[0].price}</span>
          </div>

          <div className="col-md-4">
            <img src="/img/fruit2.jpg" width="80%" />
            <h4>{fruit[1].title}</h4>
            <p>{fruit[1].content}</p>
            <span>{fruit[1].price}</span>
          </div>

          <div className="col-md-4">
            <img src="/img/fruit3.jpg" width="80%" />
            <h4>{fruit[2].title}</h4>
            <p>{fruit[2].content}</p>
            <span>{fruit[2].price}</span>
          </div>

        </div>

</div>
```

</td>
<td>브라우저에서 확인
터미널 - npm run dev
</td>
</tr>
</table>

<table>
<tr>
<td rowspan="2">3</td>
<td>src 폴더
> src 폴더안에
> components 폴더 만들어서
> Product.jsx 파일 만들기

rsc</td>
<td>

```
import React from "react";

const Product = (props) => {
  return (
    <div className="col-md-4" style={{marginBottom:"50px"}}>
      <img src={props.fruit.imgUrl} width="80%" />
      <h5 style={{marginTop:"10px"}}>{props.fruit.title}</h5>
      <p>{props.fruit.content}</p>
      <span>{props.fruit.price}</span>
    </div>
  );
};

export default Product;
```

</td>
<td>
</td>
</tr>
<tr>
<td>컴포넌트는 첫글자를 대문자로
하기로 약속했음.
첫 글자가 대문자여야
React가 사용자 정의 컴포넌트라고
인식함</td>
<td>브라우저에서 확인 : 상품잘나오는지
App.jsx에서 콘솔로 데이터 오는지 확인
let [fruit] = useState(data);
console.log(fruit[0].price);</td>
<td> 
</td>
</tr>
</table>

<table>
<tr>
<td rowspan="2"></td>
<td>props를 구조분해 할당으로 변경

Product.jsx 수정
바뀐뒤에도 동일하게 나오는지 확인</td>
<td>

```
import React from "react";

const Product = (props) => {
    const { id, title, price, imgUrl, content} = props.fruit;
  return (
    <div className="col-md-4" style={{ marginBottom: "50px" }}>
      <img src={imgUrl} width="80%" />
      <h5 style={{ marginTop: "10px" }}>{title}</h5>
      <p>{content}</p>
      <span>{price}</span>
    </div>
  );
};

export default Product;
```

</td>
<td>브라우저에서 확인
코드 변경 후 에도 상품이 잘 나오는지</td>
</tr>
<tr>
<td>App.jsx 상단에 import</td>
<td colspan="2">

```
import Product from './components/Product';
```

</td>
</tr>
<tr>
<td>4</td>
<td>App.jsx에서 수정
상품 3개를 지우고
상품 컴포넌트 map으로 코드 줄이기
map으로 상품 가져오기</td>
<td>

```
<div className="container">
  <div className="row">

    {
        fruit.map((ele,i)=>{
          return(
            <Product fruit={fruit[i]}  i={i} key={data[i].id} />
          )
        })
    }
  </div>
</div>
```

</td>
<td>

```
{
    id: 1,
    title: "수박",
    imgUrl: "img/fruit1.jpg",
    content: "당도선별 프리미엄 고당도 하우스수박 5~6kg",
    price: 29000,
}
```

<table>
<tr><td>i</td><td>0</td><td>반복 중인 인덱스 번호</td></tr>
<tr><td>fruit[i]</td><td>0</td><td>i번째 과일 객체</td></tr>
<tr><td>data[i].id</td><td>1</td><td>i번째 과일의 고유 ID</td></tr>
<tr><td>fruit={fruit[i]}</td><td>0</td><td>i번째 과일 데이터를 컴포넌트에 전달</td></tr>
<tr><td>i={i}</td><td>0</td><td>인덱스 번호를 컴포넌트에 전달</td></tr>
<tr><td>key={data[i].id}</td><td>1</td><td>컴포넌트를 위한 고유 식별값 설정</td></tr>
</table>

</td>
</tr>
</table>

5	브라우저 확인 슬라이더 아래로 상품 9개 왔는지 확인	

4. React Router 설정 / 메인 화면에 Router 만들기 / 메인 페이지 & 상세 페이지 Route, Link 연결

<table>
<tr><th>순서</th><th>작업환경</th><th colspan="2">명령어</th></tr>
<tr><td>1</td><td>리엑트에서 터미널에서 작업 끝내고 Ctrl + C
터미널에서 라우터 명령어로 깔기
깔고 나서 package.json 확인</td><td colspan="2">npm install react-router-dom</td></tr>
<tr><td rowspan="2">2</td><td rowspan="2">main.jsx
상단에 라우터 import
App 위아래로 <BrowserRouter>감싸기</td><td>

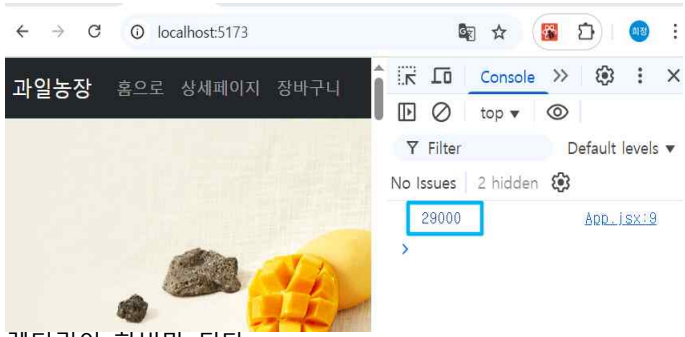
```
import { BrowserRouter } from 'react-router-dom';

  // <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  // </React.StrictMode>
```

</td><td rowspan="2">

main.jsx ×

shop > src > main.jsx

```
1
2  import { createRoot } from 'react-dom/client'
3  import './index.css'
4  import App from './App.jsx'
5  import { BrowserRouter } from 'react-router-dom';
6
7  createRoot(document.getElementById('root')).render(
8    <BrowserRouter>
9      <App />
10   </BrowserRouter>
11 )
12
```

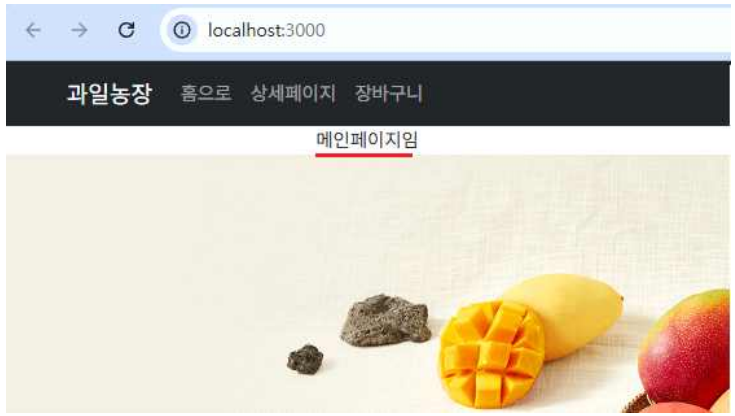
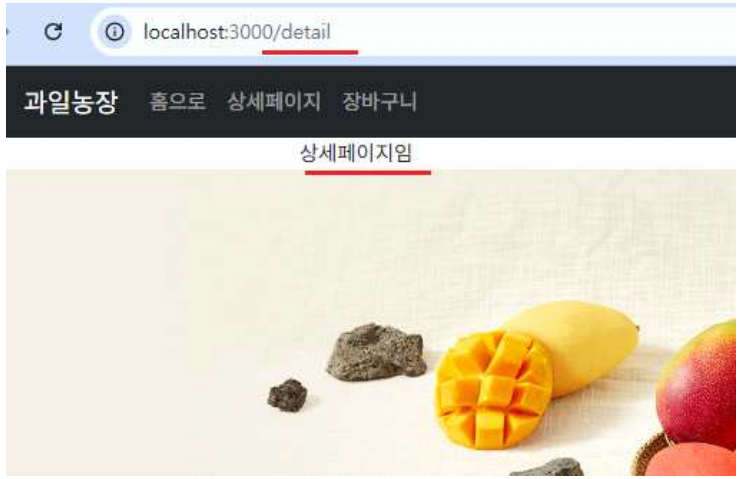
</td></tr>
<tr><td>// <React.StrictMode>를 주석으로 하면
 

렌더링이 한번만 된다.</td></tr>
<tr><td>3</td><td>App.jsx</td><td colspan="2">import { Routes, Route, Link } from 'react-router-dom'</td></tr>
</table>

<table>
<tr><td>4</td><td>App.jsx의
navbar 아래에 라우터 태그넣기</td><td><Routes>
<Route path="/" element={<div>메인페이지임</div>} />
<Route path="detail" element={<div>상세페이지임</div>} />
</Routes></td></tr>
<tr><td>5</td><td>크롬창 (로컬호스트)에서 주소 적어서 확인

라우팅(Route) 설정
Route 컴포넌트를 사용해서 경로와
연결된 화면을 설정

라우팅이란,
사용자가 요청한 URL 경로에 따라
알맞은 페이지나 컴포넌트를 보여주는 기능

메인페이지임이 중앙에 정렬이 아니면
<div className="App"
style={{textAlign:"center"}}></td><td>
<table>
<tr><td>메인</td><td>http://localhost:5173/</td><td> 
</td></tr>
<tr><td>서브</td><td>http://localhost:5173/detail</td><td> 
</td></tr>
</table>
</td></tr>
</table>

6

라우터 메인페이지임 사이에 오른쪽 코드 집어넣기

```
<Route path="/"
element={<div>메인페이지임</div>} />
```

크롬창에서 확인	
메인	http://localhost:5173/
서브	http://localhost:5173/detail

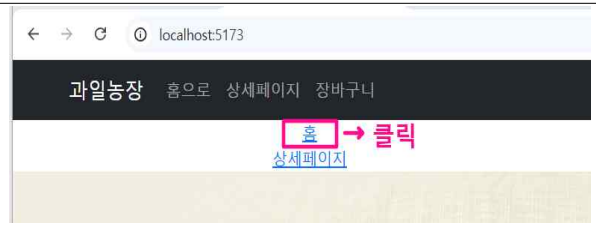
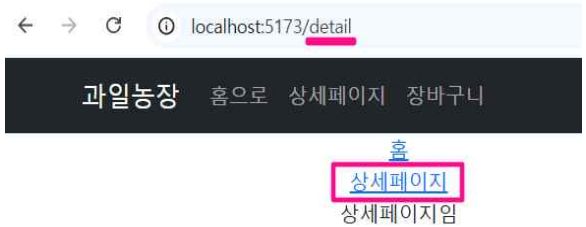
//아래의 코드를 떼서(잘라서) **메인페이지임** 사이에 **넣기**

```
<div className="slider"></div>

<div className="container">
  <div className="row">

  {
      fruit.map((ele,i)=>{
          return(
            <Product fruit={fruit[i]}  i={i} key={data[i].id} />
          )
      })
    }
  </div>
</div>
```

```
<Routes>
        <Route path="/" element={<div>메인페이지임</div>} />
        <Route path="detail" element={<div>상세페이지임</div>} />
</Routes>
```

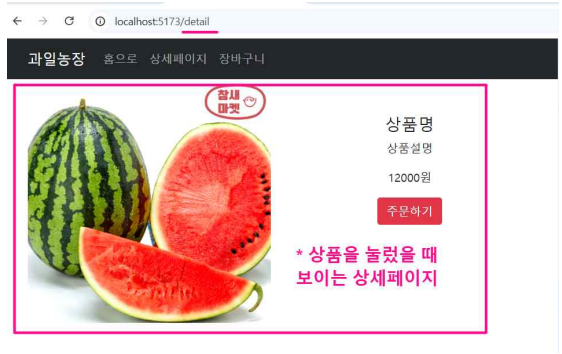
메인페이지임 자리에 슬라이더와 상품 9개를 코드를 오려내서 붙여넣는다.

		<Routes>
		<Route path="/" element={<div>
		<div className="slider"></div>
		
		<div className="container">
		<div className="row">
		{
		fruit.map((ele,i)=>{
		return(
		<Product fruit={fruit[i]} i={i} key={data[i].id} />
		)
		})
		}
		</div>
		</div>
		
		</div>} />
		
		<Route path="detail" element={<div>상세페이지임</div>} />
		</Routes>	

7	App.jsx navbar 아래 링크주소 보내기 사용해보고 주석처리(Ctrl+/) 하세요	</Navbar> <Link to="/">홈</Link> </br> <Link to="/detail">상세페이지</Link> <Routes>	
		브라우저에서 확인하고 홈과 상세페이지 클릭 홈은 경로가 / 일 때 슬라이더와 상품 9개가 보임 상세페이지는 경로가 /detail일 때 상세페이지만 보임	

5. 상세 페이지 컴포넌트 제작

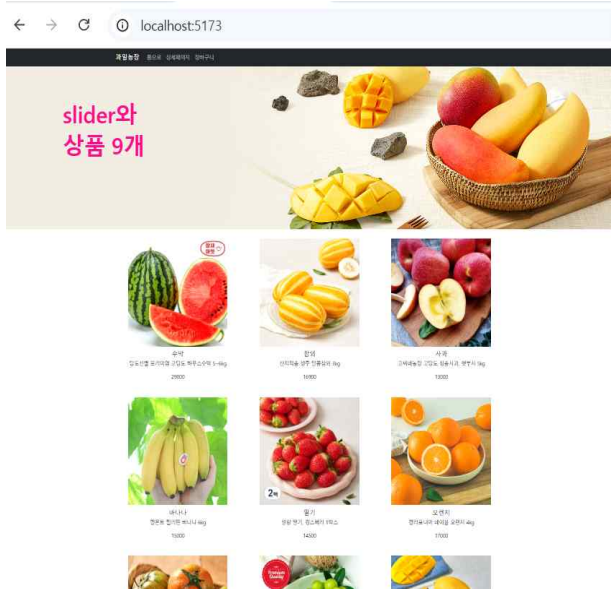
순서	작업환경	명령어	

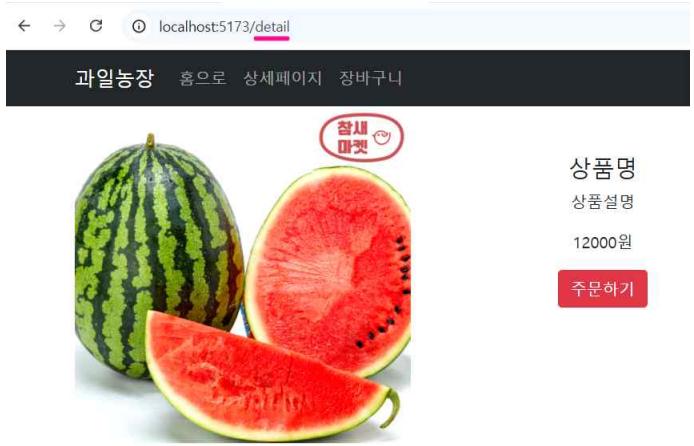
1	왼쪽폴더메뉴에서
	src 폴더
	> components 폴더
	> **Detail.jsx** 만들기
		import React from 'react';
		const Detail = () => {
		return (
		 <div className="container">
		 <div class="row">
		 <div class="col-md-6">
		<img src="img/fruit1.jpg" width="100%" />
		
		 <div class="col-md-6">
		<h4 class="pt-5">상품명</h4>
		 상품설명
		 12000원
		<button class="btn btn-danger">주문하기</button>
		
		);
		};
		
		export default Detail;
			

2	App.jsx import	import Detail from './components/Detail';	
3	<Routes>태그 사이에 <Route path="detail" element={ 상세페이지임 } /> 코드를 <Detail/> 수정	<Route path="detail" element={<Detail/>} />	

크롬창에서 직접 URL 작성 (URL로 요청)

메인	http://localhost:5173/	서브	http://localhost:5173/detail



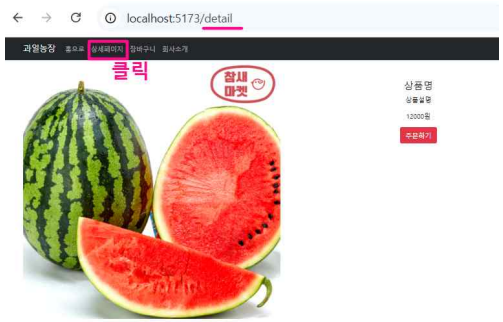


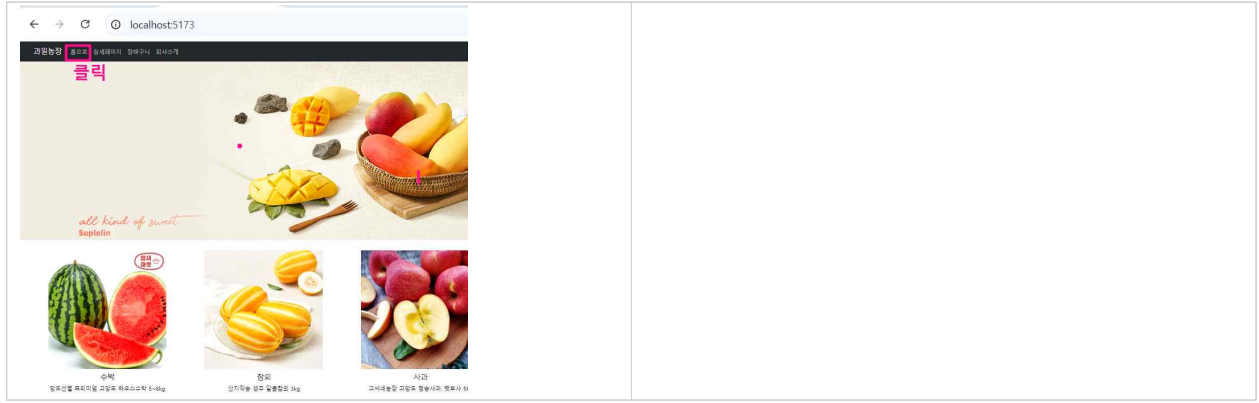
<table>
<tr><td>4</td><td>App.jsx 상단에 추가
useNavigate, Outlet 사용</td><td>상단에 있는곳 라우터 돔에서 추가하기(기존코드에서 추가된 것만 확인)
import { Routes, Route, Link, useNavigate, Outlet } from 'react-router-dom'</td></tr>
<tr><td>5</td><td>App.js 안에

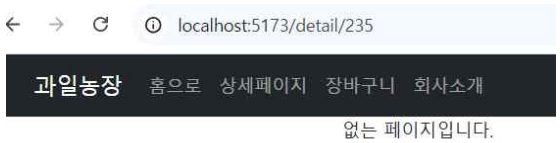
function App(){
const [fruit] = useState(data);
여기에 추가
}</td><td>const navigate = useNavigate();</td></tr>
<tr><td>6</td><td>App.js 안에

nav 태그안에 추가

Nav.Link를 이용해
페이지 이동을 구현
(네비게이션 링크 추가)

라우터를 만들고
링크를 걸어야함</td><td>
<table>
<tr><td>기존코드
파란부분을 삭제</td><td><Nav className="me-auto">
<Nav.Link href="#home">홈으로</Nav.Link>
<Nav.Link href="#detail">상세페이지</Nav.Link>
<Nav.Link href="#cart">장바구니</Nav.Link>
</Nav></td></tr>
<tr><td>빨간색 코드로 수정</td><td><Nav className="me-auto">
<Nav.Link onClick={()=>{ navigate('/')}}>홈으로</Nav.Link>
<Nav.Link onClick={()=>{ navigate('/detail')}}>상세페이지</Nav.Link>
<Nav.Link onClick={() => { navigate('/cart') }}>장바구니</Nav.Link>
<Nav.Link onClick={() => { navigate('/about') }}>회사소개</Nav.Link>
</Nav></td></tr>
</table>
<table>
<tr><td colspan="2">크롬창에서 확인 (페이지 클릭)</td></tr>
<tr><td>메인 http://localhost:5173/</td><td>서브 http://localhost:5173/detail</td></tr>
<tr><td></td><td> 
</td></tr>
</table>
</td></tr>
</table>

<table>
<tr>
<td></td>
<td></td>
<td>


</td>
</tr>
<tr>
<td rowspan="2">7</td>
<td>참고만
앞으로 한페이지 이동
뒤로 한페이지 이동</td>
<td>
<table>
<tr><td colspan="2">navigate(1) 참고만</td></tr>
<tr><td><Nav.Link onClick={()=>{ navigate(1)}}>Home</Nav.Link></td><td>앞으로 한페이지 이동</td></tr>
<tr><td><Nav.Link onClick={()=>{ navigate(-1)}}>Home</Nav.Link></td><td>뒤로 한페이지 이동</td></tr>
</table>
</td>
</tr>
<tr>
<td>404 페이지
route detail 아래에 추가
*은 전체 모두라는 뜻</td>
<td>
{/* 404페이지 */}
<Route path="detail" element={<Detail />} />
<Route path="*" element={<div>없는 페이지입니다.</div>} />
<table>
<tr><td colspan="2">크롬창에서 확인</td></tr>
<tr><td>메인</td><td>http://localhost:5173/detail/235</td></tr>
</table>
크롬창에서 확인


</td>
</tr>
</table>

8

nested routes (태그안에 태그)
Outlet (보여줄 자리를 정함)

- 여러 유사한 페이지가 필요할 때
- 모달이나 tap 페이지를 만들 때

중첩 라우트는

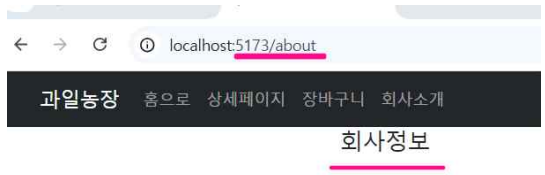
부모 컴포넌트에 <Outlet />이 있어야
자식 컴포넌트가 보임

일반적인 라우터	브라우저에서 확인
App.jsx에서 Route <Detail/> 아래에 코드 3줄 추가	

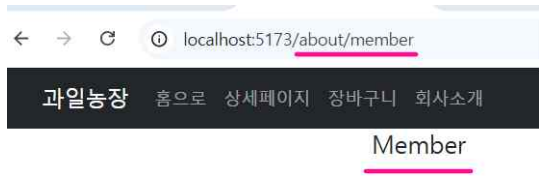
```
<Route path="detail" element={<Detail/>} />

<Route path="/about" element={<About/>} />
<Route path="/about/member" element={<Member/>} />
<Route path="/about/location" element={<Location/>} />
```

```
//app 아래, export default App; 위에
function About(){
  return(
    <>
      <h4>회사정보</h4>
    </>
  );
}


function Member(){
  return(
    <>
      <h4>Member</h4>
    </>
  );
}


function Location(){
  return(
    <>
      <h4>Location</h4>
    </>
  );
}
```

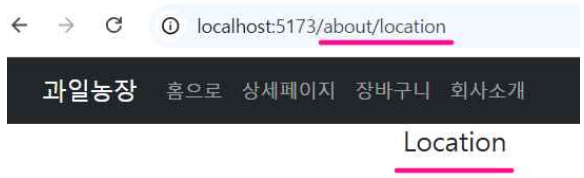
http://localhost:5173/about/location



http://localhost:5173/about/member



http://localhost:5173/about/location




```
export default App;
```

1 **nested route** (태그안에 태그가 들어감) - 2 **Outlet**

수정전	수정후
<Route path="/about" element={<About/>} /> <Route path="/about/member" element={<Member/>} /> <Route path="/about/location" element={<Location/>} />	<Route **path="/about"** element={<About/>} > <Route path="member" element={<Member/>} /> <Route path="location" element={<Location/>} /> </Route>

//app 아래, export default App; 위에
function About(){
return(
<>
<h4>회사정보</h4>
</>
);
}
	//app 아래
	function About(){
	return(
	<>
	<h4>회사정보</h4>
	<Outlet></Outlet>
	></>
	);
	}

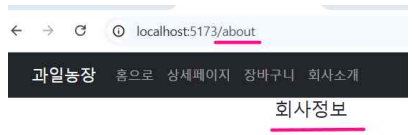
1 **nested route** **접속시**하고
2 about안에 **Outlet**을 넣어서 url 요청하면
element 2 개가보임

회사정보	/about
멤버	/about/member
위치	/about/location

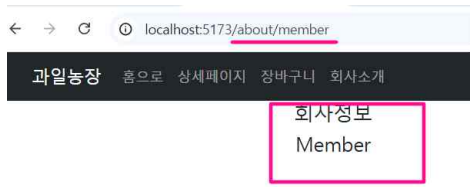
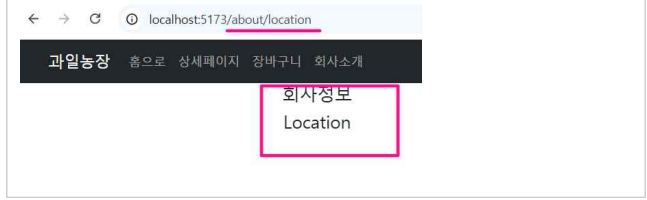
크롬확인
http://localhost:5173/about/member
about 과 member 둘다 보임
(about안에 **Outlet**을 넣어서 member를 보여주는 것임)

크롬창에서 확인

about	http://localhost:5173/about



member	http://localhost:5173/about/member



location	http://localhost:5173/about/location